

Das Vorhaben teilt sich sauber in zwei Ausbaustufen auf: eine rein passive Benachrichtigungsphase (Version 1) und eine interaktive Bot-Phase (Version 2). Der entscheidende Knackpunkt liegt darin, ob GitHub Actions einseitig Daten sendet oder ob Nutzer aktiv Anfragen stellen können.

Grundlegendes Architektur-Konzept

GitHub Actions ist ein transienter Runner (er startet, führt einen Job aus und beendet sich). Er eignet sich hervorragend, um zeitgesteuert Daten zu verarbeiten und an Discord zu schicken. Er kann jedoch **keinen dauerhaften WebSocket-Listener** betreiben, um Nutzereingaben in Echtzeit entgegenzunehmen.

Komponente	Version 1: Push-Only (Abo & Info)	Version 2: Interaktiv (Bot-Befehle)
Auslöser	<code>schedule</code> (cron) in GitHub Actions	Nutzerinteraktion (<code>/slash-command</code> , Button)
Discord-Schnittstelle	Webhook oder einfacher Bot-Token (REST API)	Vollwertige Discord Application (Bot / Interactions)
Server / Brücke nötig?	Nein, GitHub Actions sendet direkt an Discord	Ja, permanenter Server oder Serverless Endpoint nötig
Nutzererlebnis	Rolle im Discord-Server zuweisen / Channel abonnieren	Interaktion per Chatbefehl oder UI-Buttons

...

Phase 1: Die Benachrichtigungsebene (Reiner Push)

Für Version 1 benötigst du **keine externe Middleware oder Brücke**. GitHub Actions pusht die fertigen Daten direkt via HTTP POST an Discord.

1. Serverseite (GitHub & Discord)

- **Discord-Kanal & Webhook einrichten:**
 - Im Discord-Server erstellst du einen Kanal (z. B. `#updates` oder `#reports`).
 - Unter *Kanaleinstellungen* > *Integrationen* > *Webhooks* generierst du eine Webhook-URL.

- **GitHub Repository Secrets:**
 - Hinterlege die Webhook-URL als GitHub Actions Secret (`DISCORD_WEBHOOK_URL`), damit sie nicht öffentlich im Code steht.
- **GitHub Actions Workflow:**
 - Am Ende deines bestehenden Workflows fügst du einen Schritt ein, der die erzeugten Daten formatiert (als Embed oder Text) per `curl` oder über ein kleines Skript an die Webhook-URL schickt.
- **Rollenmanagement in Discord:**
 - Erstelle eine Rolle (z. B. `@Subscriber`).
 - Der Workflow kann bei neuen Nachrichten `<@&ROLLEN_ID>` erwähnen, damit nur Abonnenten eine Notification erhalten.

2. Anwenderseite

- Die Anwender treten deinem Discord-Server bei.
- **Abonnieren:**
 - Der sauberste Discord-Standard dafür ist **Onboarding & Rollen-Zuweisung** (über Server-Einstellungen > Onboarding) oder ein Kanal mit **Reaktions-Rollen** (z. B. via Reaction-Bot wie Carl-bot): Klickt der Nutzer auf ein Emoji, erhält er die Rolle `@Subscriber` .
 - Alternativ: Nutzt du einen Discord-*Ankündigungskanal*, können Nutzer mit eigenem Server den Kanal per "Folgen"-Button direkt in ihren privaten Server spiegeln.

Phase 2: Die Interaktionsebene (Bot-Interaktionen)

Sobald Nutzer mit der Anwendung sprechen oder Statusabfragen triggern sollen, stößt GitHub Actions allein an seine Grenzen:

```
[Nutzer tippt /status]
  ↓
[Discord API]
  ↓ (HTTP POST Interaction)
[Brücke / Middleware] ← (Speichert Auth/State)
  ↓ (GitHub API: workflow_dispatch)
[GitHub Actions Runner]
```

1. Warum wird hier eine Brücke benötigt?

Discord verlangt bei Slash-Commands (`/command`) oder Button-Klicks eine Antwort innerhalb von **3 Sekunden**. Ein GitHub Actions Runner braucht im Schnitt 15–45 Sekunden allein zum Starten und Bereitstellen der Umgebung.

Du brauchst daher eine schlanke Middleware, die:

1. Den Discord-Request verifiziert und sofort mit einem Status ("Wird verarbeitet...") quittiert.
2. Über die **GitHub REST API** (`POST /repos/{owner}/{repo}/actions/workflows/{id}/dispatches`) den gewünschten Workflow anstößt.
3. Später das Ergebnis über Discord Webhooks oder die Interaction-Token-URL an den Nutzer zurückliefert.

2. Geeignete Optionen für die Brücke

- **Serverless Edge Function (kostengünstig / wartungsarm):** Cloudflare Workers, AWS Lambda oder Vercel Serverless Functions. Discord unterstützt seit längerem *HTTP Interactions*, sodass du keinen dauerhaft laufenden Bot-Prozess über WebSockets brauchst, sondern reine HTTP-Endpunkte nutzen kannst.
- **Schlanker Container-Host:** Falls du WebSockets (discord.js / discord.py) bevorzugst: Fly.io, Railway oder ein kleiner VPS (z. B. Hetzner).

3. Anwenderseite in Phase 2

- Nutzer tippen Slash-Commands (z. B. `/trigger-check` oder `/abo-status`).
- Alternativ klicken sie auf Buttons direkt unter den Benachrichtigungen, um Details anzufordern.

Zusammenfassende Empfehlung für den Start

1. **Starte schlank ohne Brücke:** Richte in Discord Rollen ein (z. B. via Server-Onboarding) und lass deinen bestehenden Cron-Workflow in GitHub Actions am Ende via HTTP-POST an einen Discord-Webhook senden.
2. **Entkopple die Logik:** Trenne die Datengenerierung in deinem Skript von der Discord-Formatierung, damit du später problemlos von einem einfachen Webhook auf eine vollwertige API-Payload umstellen kannst.
3. **Erweitere erst bei echtem Interaktionsbedarf:** Plane für Phase 2 eine serverlose Funktion (z. B. Cloudflare Worker mit `workflow_dispatch`), um Kosten und Serverwartung minimal zu halten.

Soll ich dir ein Beispiel für den curl/Webhook-Schritt in GitHub Actions zeigen?

↔ Ja